jamk

From Modular Monolith to Microservices

Design and Implementation of a Distributed Event Ticketing System with .NET

Aleksandr Makarov

Bachelor's thesis
June 2026
Information and Communication Technologies

jamk | Jyväskylän ammattikorkeakoulu
University of Applied Sciences

jamk

Description

Aleksandr Makarov

From Modular Monolith to Microservices: Design and Implementation of a Distributed Event Ticketing System with .NET

Jyväskylä: Jamk University of Applied Sciences, June 2026, 48 pages.

Degree Programme in Information and Communication Technologies. Bachelor's thesis.

Permission for open access publication: Yes

Language of publication: English

Abstract

Application Architecture is quite important during the development; the performance and manageability of a system are heavily dependent on the team's productivity and the architecture the system is designed under. However a number of companies build their application from the beginning with microservices, which will create high difficulties to the team while develop at the beginning, as you need to make sure data is always consistent. Meanwhile, monolithic architecture at later stage causes complexity to application deployment, since all components have strict dependency on each other, it's quite hard to extract a single component from the system. As the result, during the early stage we need to find out a method to migrate from monolith architecture to microservices without complicating work. Hence we had designed an application: event ticketing system, which is an application shows how to transit from modular monolith to microservices architecture. It contains techniques of how to decouple microservices, using queues, testing and deployment.

The application was implemented with a design approach based on research. The application has been developed with the .NET framework and ASP.NET Core. It was developed with a modular architecture, which enables each module to spin off into separate microservices at a later stage. Each module is composed of a domain layer, a data access layer and a presentation layer. Modules communicate via public APIs and queues using the publish-subscribe pattern. Basic information is stored in a database; the finished application is containerized and can be deployed on any system.

As a result of the thesis project, an application for booking event tickets was developed. To evaluate the results of the work, feedback was gathered through an analysis of technical comparative data and development time, as well as a user survey using a questionnaire. The evaluation of the results showed that the transition from monolithic architecture to microservices has a positive impact on the speed and simplicity of application development.

Keywords/tags (subjects)

Modular Monolith, Microservices Architecture, Distributed Systems, Event-Driven Architecture, .NET, ASP.NET Core, Domain-Driven Design, Scalability, Fault Tolerance, Cloud-Native Applications

Miscellaneous (Confidential information)

-

Contents

1 Introduction 4
1.1 Background 4
1.2 Problem Definition 4
1.3 Research Objectives and Questions 5
1.4 Scope and Limitations 5
1.5 Ethics 6
2 Theoretical Framework 6
2.1 .NET Framework 6
2.2 Software system architecture 7
2.2.1 Monolithic architecture 7
2.2.2 Microservices architecture 8
2.3 Monolith vs Modular Monolith vs Microservices 9
2.4 When Modular Monoliths Fail 9
2.5 Messaging and Event-Driven Architecture 10
2.6 Migration from Monolith to Microservices 11
2.7 Modular Monolith as a Transition Architecture 11
3 Methodology 12
3.1 Research-based design 12
3.2 Workflow 13
3.3 Data Collection 13
3.4 Development Tools and Frameworks 14
3.5 Validity and Reliability 15
4 System Architecture and Design 16
4.1 Architectural Overview 16
4.1.1 Modular Decomposition 16
4.1.2 Layered Architecture 16
4.1.3 Domain Layer 17
4.1.4 Application Layer 17
4.1.5 Infrastructure Layer 17
4.1.6 Presentation Layer 18
4.1.7 Benefits of logic separation 18
4.2 Data Integration 19
4.3 Communication Between Modules 19

4.4 Scalability 20
4.5 Why Modular Monolith Instead of Traditional Monolith or Microservices? 20
5 Implementation 22
5.1 Users Module 23
5.1.1 User Registration 24
5.1.2 Email confirmation 25
5.1.3 Login without password 25
5.1.4 Integration with Notification module 27
5.1.5 Analytics of architectural solution 27
5.2 Scheduling module 27
5.2.1 Configuring event types 28
5.2.2 Calculating availability of slots 28
5.2.3 Working with Time Zones 29
5.2.4 Synchronization with Booking Module 30
5.2.5 Analytics of architecture solutions 30
5.3 Booking Module 31
5.3.1 Booking Workflow 32
5.3.2 Analytics of architecture solutions 33
5.4 Notifications Module 33
5.5 Evolution from Modular Monolith to Microservices 34
5.5.1 Initial Architecture: Modular Monolith 34
5.5.2 Introduction of Event-Driven Communication 35
5.5.3 Extracting the Booking Module into Microservice 36
5.5.4 Introducing an API Gateway 37
6 Evaluation and Results 38
6.1 Scalability Analysis 38
6.2 Architecture analysis 39
6.3 Survey analysis 39
6.4 How does a modular monolith help to structure a system before transitioning to microservices? 41
6.5 What principles are used to identify modules with clearly defined responsibilities? 42
6.6 How is the phased migration of an individual module to a microservice carried out? 42
6.7 What challenges arise when transitioning partially to a microservices architecture? 43
6.8 How is interaction between the monolithic and microservice parts of the system ensured? 44
6.9 Overall assessment of the results 44

7 Conclusion 45

7.1 Reflection 45

7.2 Practical Recommendations 45

7.3 Future Work 46

References 47

Figures

Figure 1. Bookify Solution 23

Figure 2. Users entities 24

Figure 3. User registration flow 24

Figure 4. Email confirmation flow 25

Figure 5. Request login link flow 26

Figure 6. Sign in with login link flow 26

Figure 7. Scheduling entities 28

Figure 8. Slot availability flow 29

Figure 9. Synchronization flow 30

Figure 10. Booking entities 31

Figure 11. Booking creation flow 32

Figure 12. Booking payment flow 33

Figure 13. Notification flow 34

Figure 14. Initial modules 35

Figure 15. Event driven communication 36

Figure 16. Module extraction 37

Figure 17. Gateway introduction 38

Figure 18. Multiple Booking API instances 38

Tables

Table 1. Tools 14

Table 2. Architecture comparison 21

1 Introduction

1.1 Background

In today's world, web applications are developed against a backdrop of constantly expanding codebases, increasingly complex business requirements, the integration of new modules, and the duplication of services to handle massive workloads, whilst multiple teams may be working on a single project. Under such conditions, the system must evolve. Traditional monolithic applications eventually face scalability issues and deployment challenges, whilst the transition to microservices architecture often leads to a highly distributed application. Consequently, an intermediate architectural approach has emerged in the form of a modular monolith, which ensures strict system modularity and prepares it for the smooth separation of individual microservices.

1.2 Problem Definition

As the codebase grows, working on a monolithic application becomes difficult, as deployment times, the process of making changes, and code testing all take longer. The transition from monolithic architecture to microservice architecture is quite challenging, as all services in a monolith are tightly coupled, making it difficult to isolate individual modules. On the other hand, if you start developing an application using a microservices architecture from the outset, issues arise regarding data consistency, as well as the complexity of debugging and testing. Microservices architecture may require an entire team of developers.

A key factor is the absence of an intermediate architectural layer between the monolith and the microservices. Often, the transition occurs directly, which usually leads to a significant increase in system complexity, without any adaptation of the development team to a distributed architecture. Consequently, situations arise where the boundaries between services are not clearly defined. Such situations usually result in microservices remaining highly interconnected and dependent on one another, much like in a monolithic system. In this regard, the modular monolith is one approach to migration, allowing for the identification of independent modules with their own control boundaries, whilst maintaining a single point of deployment.

1.3 Research Objectives and Questions

The aim of this thesis work is to demonstrate the transition from a monolithic system to a microservices architecture using a modular monolith. An event ticketing application will be used as an example, as it contains modules with distinct boundaries. Each module will be spun off into a separate microservice.

To achieve this objective, the thesis must answer the following questions:

1. How does a modular monolith help to structure a system prior to migrating to microservices?
2. What principles are used to identify modules with clearly defined responsibilities?
3. How is the phased migration of an individual module to a microservice carried out?
4. What problems arise during a partial transition to microservices architecture?
5. How is interaction between the monolithic and microservices parts of the system ensured?

This study aims not only to develop a prototype system, but also to analyze the architectural approach involved in transitioning from a monolithic system to a microservices-based one. It includes an assessment of the impact of modularity on scalability, maintainability and development complexity.

1.4 Scope and Limitations

This thesis focuses primarily on providing a modular monolithic architecture solution and a prototype web application. The thesis does not aim to develop a fully-fledged enterprise-grade application but rather uses the application as an example to illustrate the development process. The idea is to develop a prototype on which all the same development stages inherent in a real application can be carried out, from module design and boundary definition to testing and deployment.

The technical part of the thesis focuses on:

1. Architecture: Design a modular monolithic application with distinct modules and access via public APIs. Identify one module that will subsequently be spun off into a separate microservice.
2. Prototyping: Develop a working prototype of a web application using .NET and ASP.NET Core based on modular architecture.
3. Evaluation: evaluate the results by comparing the system before and after modularization according to the following criteria: coupling, cohesion, separation of concerns and modifiability. Code complexity metrics and dependencies between components are used for quantitative assessment.

The development has its limitations; the application has been tested exclusively in a prototype environment and has not been subjected to real-world high-load conditions. The evaluation of architectural solutions is based on qualitative analysis, observations made during development, and a limited set of test scenarios. Usability assessment and feedback collection were conducted on a limited scale. Despite the limitations listed above, this work demonstrates a practical approach to using a modular monolith as a transitional step between a traditional monolithic system and a distributed microservices architecture.

1.5 Ethics

In this thesis work, artificial intelligence is used as a supporting tool. OpenAI's ChatGPT is used to check compliance with APA 7 style guidelines, search for information, and seek advice on coding issues.

Personal data was collected or processed during the study. The system developed is used exclusively as a prototype for research purposes and is not intended to store actual user data. The development process considered the fundamental principles of responsible software engineering. Randomly generated links were used for user authentication to eliminate the need to store passwords and other personal information.

2 Theoretical Framework

2.1 .NET Framework

Developing modern web applications often requires platforms that support scalability, flexibility, and enable deployment across multiple operating systems. Microsoft (n.d.) states that .NET is an open-source platform for developing cross-platform applications of various types. The most popular development language is C#, but this doesn't limit development in other languages. A high-performance runtime environment is used for developing high-scale production apps. Such platforms often enable developers to use a single framework and select different programming languages and architectural approaches based on the project requirements. One of the popular products for web application development is ASP.NET Core. It's an open-source, cross-platform development

framework that can handle large volumes of work, making it a robust choice for enterprise-level applications (Microsoft, n.d.). Based on these characteristics, .NET and ASP.NET appear to be widely adopted by developers in both commercial and enterprise software projects, where reliability and manageability are of critical importance.

2.2 Software system architecture

2.2.1 Monolithic architecture

It is essential to understand the differences between software architecture approaches as they directly affect the maintainability, scalability, and efficiency. Due to its ease of development and management, monolithic architecture is becoming a top choice when deciding on business application development (Blinowski et al., 2022). “Enterprise applications are often internally built according to the classic three-tier model and hence consist of: (1) user interface code …; (2) server-side business logic which handles HTTP requests, executes domain logic” (Blinowski et al., 2022, p. 2). Monolithic architecture is a software system designed as centralized system on a tiered model. In such a system, all components are typically tightly integrated. Development, testing, and deployment are often carried out as a single unit for the entire application. This simplifies code management and version control, as all functionality is maintained within a single repository.

The choice of software architecture significantly influences the system’s behavior underload, its ability to evolve over time, and interaction patterns within development teams. A monolithic architecture runs on a single process, which in turn manages all the code. Such an application is easy to deploy, but also difficult to scale and update, especially when multiple teams work on the same project. On the other hand, a monolithic application is easier to maintain because it uses centralized database, and all application modules work as one unit when deployed. Unlike a traditional monolith, a distributed monolith consists of several individual modules, but they are tightly coupled and must be deployed together (Newman, 2019). These parameters highlight why monolithic applications tend easier to develop and are often used in the initial phase, whereas complex systems may require a transition to alternative architectural approaches.

2.2.2 Microservices architecture

As system grows it becomes necessary to translate to an alternative architecture, such as microservices, as it allows the application to be decomposed into smaller services and each component to be deployed independently. Fowler and Lewis (2014) introduced the concept of microservices to address challenges in monolithic architectures. “In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API” (Fowler & Lewis, 2014, para. 3). The ability to divide a system into microservices allows developers to use a different programming language for each service and allows them to select technologies based on the tasks or development requirements (Fowler & Lewis, 2014). This architectural approach improves modularity and allows teams to work on services independently.

The choice of how finely to decompose a system into services is a key factor in microservices architecture. For example, if services are divided into too large pieces, the system loses its key advantages: scalability, loose coupling, and testability. On the other hand, if services are too finely divided, system orchestration, application deployment, and data synchronization become difficult (Newman, 2019). As the number of services in a system increases, its complexity increases. An application in microservice architecture is made of many tiny, loosely coupled parts, but they need to work as a single system. (Newman, 2019). The balance between service granularity and system complexity is a key design consideration the implementation of microservices-based architectures in real-world applications.

Communication between services plays a vital role in system productivity and scalability. “One of the most popular ones is that every service publishes a REST API over HTTP … For asynchronous communication, using messaging instead of REST APIs provides better performance and reliability. Message brokers like RabbitMQ can provide message queues” (Kalske, 2017, p. 26). Synchronous communication returns a response immediately and works by sending requests through an HTTP client to another service. This ensures tighter coupling, since if one service stops working, the other will not receive a response. Asynchronous communication uses a message broker, such as RabbitMQ, which provides looser coupling and increases fault tolerance, since a message can be

processed after a service restart (Kalske, 2017; RabbitMQ, 2026). The choice between synchronous and asynchronous communication depends on system requirements and factors such as latency sensitivity, fault tolerance, and the acceptable levels of coupling.

2.3 Monolith vs Modular Monolith vs Microservices

To meet the organization's challenges of achieving efficiency in the system's development with long-term changes, various architectural styles such as monolithic, modular monolith and microservices architectures were derived. Al-Debagy and Martinek (2018) argue that monolithic and microservice architectures are two different approaches for decomposing systems. A monolithic system is a system that encapsulates all functionalities into a single executable file, single deployed instance. A microservice system is a system that encapsulates all functionalities into several services that could be deployed independently. The modular monolith is somewhere in the middle, it keeps the single deployable instance of the traditional monolithic system, but defines well-decoupled modules for each responsibility, so it aims to be simpler than microservice and more maintainable than monolith.

Both have significant implications for software development and maintenance. A monolithic architecture offers ease of deployment and reduced operational costs and overhead. Therefore, it is appropriate for smaller or medium-sized applications. As the application grows, the strong coupled nature between the services will cause more maintenance costs and decrease the development velocity. While microservice architecture will gain benefits such as easy scalability and separate service deployment yet have the disadvantage of adding much more complexity in terms of service communication, monitoring, data consistency, infrastructure maintenance and so on.

2.4 When Modular Monoliths Fail

Despite being a good architectural transition phase, modular monoliths don't fit everywhere. Bad defined boundaries between modules may become a distributed monolith inside the application itself. Also, if the team wants to scale/deploy separate business areas, the modular monolith architecture will no longer work. Newman (2020) argues that architectural boundaries must be carefully maintained within a modular system; otherwise, increasing dependencies between modules can gradually transform the application into a tightly coupled monolith.

This issue is commonly referred to as boundary erosion. When these internal modules no longer only depend upon interfaces but develop a direct dependence on other modules internal architecture, the separation between the modules begins to deteriorate slowly over time. This can make maintaining the system, testing it, and modifying it far more challenging, and after time, the architecture can lose most of the benefits that are gained from a modular application and begin to act like an old-style monolithic application (Newman, 2020).

2.5 Messaging and Event-Driven Architecture

Effective communication between services is essential in distributed systems, as data must be transmitted over reliable channels. Business logic in messaging systems is processed by consumers, which in turn receive messages from senders. Message queues are a common approach for exchanging data in distributed systems. Message queues are often used in interposes and inter-thread communication, and they can also act as a message bus for transmitting information in distributed systems. The basic principle of a queue is to adhere to the "first in, first out" model, meaning it uses a linear data structure (Kleppmann, 2017). This structure defines the precise order of message processing, which is crucial for maintaining consistency and predictability in distributed systems.

The publish-subscribe messaging pattern is commonly adopted in distributed systems to support extensibility and decoupled communication between services. Publish-subscribe is one of the key principles for maintaining communication in messaging systems with topic- or content-based filtering. Microservice applications typically employ systems in which consumers subscribe to topics and producers publish messages to them, ensuring that messages are delivered only to the intended recipients (RabbitMQ, 2026). To ensure message delivery to the appropriate services, these messages are routed to all subscribing consumers. RabbitMQ, an open-source message broker, is often used for broker-based messaging (RabbitMQ, 2026). By communicating through shared topics rather than direct point-to-point connections, the system is flexible and can be scaled horizontally.

2.6 Migration from Monolith to Microservices

As software applications increase in complexity and user base, there's a greater shift from monolith systems to microservices architecture. According to Faustino et al. (2024), migrating a monolith to a set of microservices is primarily a need to achieve scalability, and a need to enable several development teams working independently on different parts of the application. The authors point out however, that decomposing a monolith into a set of microservices will add more complexity regarding service communication, performance overhead and the migration effort itself.

The problem of migration has another difficulty that architecture decomposition often cannot be done instantly. In big system there exist closely coupled components, shared databases, business logic that complicate the direct migration due to risk and cost. When services are split, we have a new kind of problem such as network communication, distributed transaction, data consistency, monitor, fault tolerance, etc. That increase the complexity of operating system rather than a monolithic one.

Faustino et al. (2024) further observed trend where organizations start with a modular monolith as a stepping stone towards a microservices architecture. This involves strong separation of modules while keeping everything in a single deployable unit. This helps teams to incrementally loosen the coupling of the business domains and position it for service extraction without being introduced into the world of distributed systems immediately.

The findings of Faustino et al. (2024) suggest that migration should not be done as a binary shift from monolithic to microservices architecture. A modular monolith may be very good intermediate architecture, providing both ease of maintenance and operation as well as scaling capabilities.

2.7 Modular Monolith as a Transition Architecture

The benefit of this method is that organizations can gain more maintainability and be ready for future evolution without suffering from operating distributed systems.

According to Moraes et al. (2025), modular monoliths integrate the ease of deployment from a monolith while leveraging the modular design approaches seen in microservices. According to the

authors, well-defined modules can diminish the amount of coupling between business domains, thereby improving architectural adaptability in the future.

A central concept which helps a modular monolith to work is Separation of Concerns. Each module is responsible for a business capability and exposes functionality via interfaces that define it clearly. The modules thus only have loosely coupled dependencies with other modules, and it is unlikely that changes in one place of the application will impact on features elsewhere. It makes the application more maintainable, testable, and extensible.

3 Methodology

3.1 Research-based design

This thesis is based on research-based design methodology. It combines research activities and the design process with the development of a software solution. The main objective of this research, in addition to a theoretical examination of architectural approaches, is to create a practical solution that demonstrates a gradual transition from a monolithic system to a microservices architecture.

The study was divided into several stages:

1. Analysis phase: we need to investigate existing architectural approaches to application development. These include monolithic systems, modular monoliths and microservices architecture. We also need to analyze issues relating to scalability, application maintenance and development, and interactions between system components.
2. Design and development phase: design a modular application architecture, dividing the system into separate bounded contexts and independent modules. Implement the modules using .NET and ASP.NET Core and integrate synchronous and asynchronous communication between modules using HTTP requests and queues. Isolate one of the modules into a separate microservice to demonstrate a gradual transition.
3. Evaluation phase: test and analyze the developed prototype. Assess the system's modularity, the interaction between components, and the complexity of isolating a single module into an independent microservice compared to a traditional monolithic system.

The research questions posed in this thesis require architectural solutions to be tested in practice within a real-world system. It is particularly important to determine how a modular monolith can help prepare a system for migration, what principles can be applied when isolating modules, and

what challenges may be encountered when transitioning to microservices. It is important to understand how to ensure interaction between components after the system has been split. Answering these questions based solely on an analysis of technical literature can be a difficult task, as many important implementation details only emerge during the design and development of the application.

3.2 Workflow

Work on the thesis was carried out in stages, following a research-based design approach. In the initial stage, an analysis of the subject area and existing approaches was conducted. Attention was paid to issues of scalability and component connectivity in traditional monolithic systems.

Following the analysis phase, the key functional and non-functional requirements were identified. Functional requirements include the ability for users to create accounts, manage events, book tickets and process payments. Non-functional requirements include low coupling between components, the ability to scale individual modules, support for asynchronous communication and ease of testing.

The next stage involved implementing the application prototype. During development, individual modules were implemented and integrated into stages. Each module was developed in isolation from the others, communicating via public APIs.

Following the completion of the implementation, the system was tested and the architectural solutions were analyzed. An assessment was carried out of the application's modularity, the complexity of interactions between components, and the feasibility of a future transition to a microservices architecture.

3.3 Data Collection

To address the questions raised in the thesis, the following methods of data collection were employed regarding ethics:

1. Technical analysis and comparison: data was collected by analyzing the application's structure, interactions between modules, and the complexity of deploying and integrating components. During the development process, the following parameters were assessed: the degree of coupling between modules, separation of concerns, the complexity of interactions between services, the scalability of individual parts of the system, and the ease of testing and modifying the application.
2. Survey: To further evaluate the proposed architecture, a short survey was conducted among several developers with experience in backend development. Participants were provided with the module code and the key principles governing the interaction between system components. After reviewing the material, participants answered questions regarding code readability, the clarity of the module structure, and the complexity of maintaining the system.

The results obtained were used as data for analyzing the advantages and limitations of the chosen architectural approach.

3.4 Development Tools and Frameworks

The tools listed in the Table 1 were selected for development.

Table 1. Tools

Name	Function	Description
C# and .NET	The primary platform for application development	The C# programming language and the .NET platform are used to implement the server-side of the application, the business logic and the modular architecture.
ASP.NET Core	Web application and API development	ASP.NET Core is used to create REST APIs, handle requests and manage communication between system components.

Visual Studio	Development environment	Visual Studio is used as the primary environment for developing, writing, debugging and testing software code.
External Libraries (Entity Framework, Mass Transit, MediatR)	Interaction with external services	External libraries for database interaction, organizing event-driven communication, and reducing component coupling.

3.5 Validity and Reliability

The reliability of the research is ensured using stable and widely adopted technologies. Containerization simplifies deployment and enables the creation of an environment that runs seamlessly across different operating systems. Furthermore, reliability is enhanced by modular architecture and the separation of responsibilities between components. Unit and integration tests enable errors to be detected when changes are made to the system. Modules can operate independently of one another, and the queue allows messages to be reprocessed in the event of a failure in one of the modules.

To enhance the validity of the results, the study was based on both theoretical analysis and the practical implementation of a prototype application. The findings were compared with the characteristics of traditional monolithic architecture and with results described in academic literature. Additionally, the quality of the architectural solution was assessed by analyzing the responses of the developers who took part in the survey.

The results enable an assessment of the practical applicability of the modular monolith architecture and highlight its key advantages and limitations when transitioning from a monolith to microservices.

4 System Architecture and Design

4.1 Architectural Overview

4.1.1 Modular Decomposition

One of the major problems with traditional monolithic architecture is that the interdependence of components increases as the codebase grows. Although development may seem straightforward in the early stages, over time it becomes difficult to modify parts of the system without affecting numerous other components. This complicates both maintenance and further development of the system.

To address this issue, research was conducted and modular monolith architecture was selected. This differs from a traditional monolith where business logic often lacks clear boundaries and is concentrated in a single location in that it divides the system into independent functional domains. Each domain has its own area of responsibility and a minimal number of dependencies on other modules.

This approach retains the advantages of monolithic application. There remains a single deployment process, the infrastructure is not complicated, and there are no network delays. Interdependencies between components are reduced, which prepares the system for a potential transition to a microservices architecture.

4.1.2 Layered Architecture

Dividing a system into modules does not guarantee a loosely coupled and highly maintainable system. Even though there are well-defined module boundaries, the same problem could occur within each module as with a monolith, i.e., mingling business logic with infrastructure code and user interface code.

For this reason, the investigation not only adopted modularity but also multi-layered architecture within each module. If the module architecture defines how the functional areas of a system separate, the multi-layered architecture explains the internal structure of each module.

This lower coupling both inter and intra module and makes future development and possible single module migration to separate microservices much easier.

4.1.3 Domain Layer

This layer comprises the business rules and the fundamental entities in the domain. The primary objective is to eliminate technical concerns from business logic. This ensures that the system behaves according to business rules and doesn't depend on the specific database, messaging system, or even user interface employed.

In a modular monolith this contributes to creating boundaries between modules. Each module has its own domain model and has no dependency on the internal models of other modules. This follows principles of domain-driven-design and makes it easy later to split off a module into its own microservice.

4.1.4 Application Layer

The Application Layer defines what business scenarios are exposed by the system and orchestrates the calls to be made to the domain objects and infrastructure services.

Isolating the business scenarios within a separate layer avoids the typical problem with monolithic architectures where the business logic is distributed between the controllers, the database and helper services. Each scenario has a well-defined entry point and can be tested in isolation from the infrastructure components.

The Application Layer also provides clear boundaries for later APIs. Most of the commands and requests issued from a module to a microservice can be transformed into the relevant HTTP endpoints or message handlers, with minimal changes to the business logic itself.

4.1.5 Infrastructure Layer

The infrastructure layer is concerned with the technical dependencies required for the system and this could involve the database, file storage, message queuing and any other external services (APIs) that we are communicating with.

The advantage of isolating this infrastructure code within a separate layer means that we decouple the business logic of the application from the technologies it depends on. This means that we can switch from using PostgreSQL to another DBMS or switch the message queuing service from RabbitMQ to another one without having to change any of the Domain layer or the Application layer. This becomes extremely useful from a microservices architecture migration perspective when moving functionality between services doesn't require us to completely rewrite our application logic.

4.1.6 Presentation Layer

The Presentation Layer is the way the application interacts with external users and services through an HTTP API. The Presentation Layer receives requests from the user/external services, validates it and hands over control to the Application Layer.

One of the key principles is that the Presentation Layer has no business logic in it. By not allowing business logic here we minimize coupling to the user interface. Because of this the application can have different entry points to it without any need to modify the business logic.

4.1.7 Benefits of logic separation

By applying multi-tier architecture together with a modular structure of the application, we solved two problems at once: reduced coupling between modules within the application; architectural separation of modules to prepare for future extracting modules into microservices. Layer separation isn't only organizational. It also provides the opportunity to separate business logic from technology. That makes it possible to copy the most business logic, while making most of changes on infrastructure and communication layer, when later spin off a module to separate microservice.

Introduced boundaries were essential to later extract modules into separate microservices. Therefore, the multi-layer architecture functions not only as a tool for organizing code but also as a tool for creating an evolutionary process from monolithic architecture to distributed architecture.

4.2 Data Integration

Instead of managing various types of data with one storage, it was decided to implement a polyglot persistence model where different types of data are managed by specialized tools.

In traditional monolith application one might end up managing all system data using a single relational database, which although might be faster at the beginning, will turn out inefficient and a lot harder to scale individually in the long run.

PostgreSQL has been selected as the primary data store because it supports ACID properties and transactions. It is very important in booking and payment processes where wrong transaction can cause data inconsistency. Also, relational models fit well storing related system entities.

Redis is being used to store temporarily and cache data. Storing the data in memory provides faster access time to commonly accessed entities and reduces pressure on the primary database. Also, it supports TTL, so the stale data is automatically removed without any additional application logic.

Having several dedicated storage systems has enabled the distribution of concerns for each data type, which aligns with the broader vision of modular architecture and facilitates the eventual breakdown of system into independent services.

4.3 Communication Between Modules

If all interactions between modules are handled using synchronous calls, then the system starts acting like a distributed monolith. The system business process can halt in another section of the system if one module in the system fails.

It is also not appropriate to use fully event-driven architecture. In most business operations the response time for a user's request must be prompt and solely dependent on asynchronous communication results in complex user-flow and higher response times.

From the evaluation above, the system decided to use a combination of synchronous and asynchronous communication. Using both allows us to obtain the pros of both methods while eliminating the cons of each.

Event-driven interaction is useful in preparing the system to be transitioned into microservices. When modules communicate through events, they do not anymore depend on the internal logic of other modules, they start to only communicate through contracts. In future such contract can be directly used for each separate service.

4.4 Scalability

One of the most cited benefits of the microservices architecture is individual scalability of system components, but this benefit only comes into play when the loads are unevenly distributed across the components. For most small and medium-sized projects, microservices architecture applied too early brings an increase in the infrastructure, but not the actual benefits. That's why a modular monolith pattern was applied as a transitional phase to evolve the architecture.

A modular monolith compared to a standard monolith allows pre-definition of boundaries for the future services. When load becomes too uneven in one component, or the component requires its own lifecycle, it's possible to extract it into a separate service, with no need to rewrite the system. However, a modular monolith doesn't provide full scalability as microservice architecture does. Until a module is separated, it is deployed in the same instance, sharing computing resources, as the other modules of the application. As a result, modular monoliths are not a replacement to microservices architecture but rather an architecture which takes a middle ground between the Monolith and the distributed system architectures.

4.5 Why Modular Monolith Instead of Traditional Monolith or Microservices?

Three architectural styles were taken into consideration; traditional monolithic, modular monolithic and microservice architecture. As can be seen on Table 2 each architecture has advantages and disadvantages that should be evaluated in relation to project scale, team size, and system needs.

Table 2. Architecture comparison

Criteria	Traditional Monolith	Modular Monolith	Microservices
Initial development speed	High	Medium	High
Operational complexity	Low	Medium	High
Supportability	Decreases as the system grows	High	High
Independent deployment	No	Limited	Yes
Scalability	Limited	Medium	High
Migration readiness	Low	High	High

The modular monolith is a tradeoff between monoliths simplicity and the flexibility offered by microservices architecture. The benefits include better separation of concerns, low coupling and a simpler system to manage as the codebase increases.

The modular monolith also helps avoid the high infrastructure complexity usually associated with microservices. A microservices system has distributed interaction, fault, monitoring and individual deployment mechanisms which can be costly to set up small to medium sized systems. An extra benefit of modular monolith systems is the opportunity of a staged migration toward microservices.

But then the modular monolith is not suited for all problems. When the load grows tremendously or independent teams of developers working on the system grows, then a modular monolith does not scale anymore independently for every feature or every development team and maybe a shift toward micro services would be more legitimate. Thus, the modular monolith must not be seen as the counterpart of microservices but as the in-between solution.

5 Implementation

Following the architectural vision set out in the previous chapter, the system was designed as a modular monolith, in which the system is divided into several self-contained modules based on functionality. Each module has a specific domain and is responsible for reducing the degree of coupling between system components.

In developing the prototype four modules were constructed:

1. Users - User management, registration and authentication
2. Scheduling - schedule management and available time slots
3. Booking - creating and managing bookings
4. Notifications - sending notifications and emails

Even though modules on Figure 1 are deployed as one single application, they interact with each other through explicit contracts and events/integrations. By following this methodology, it is easy to achieve low coupling between the modules, and the system is ready for eventual extraction of components as standalone microservices.

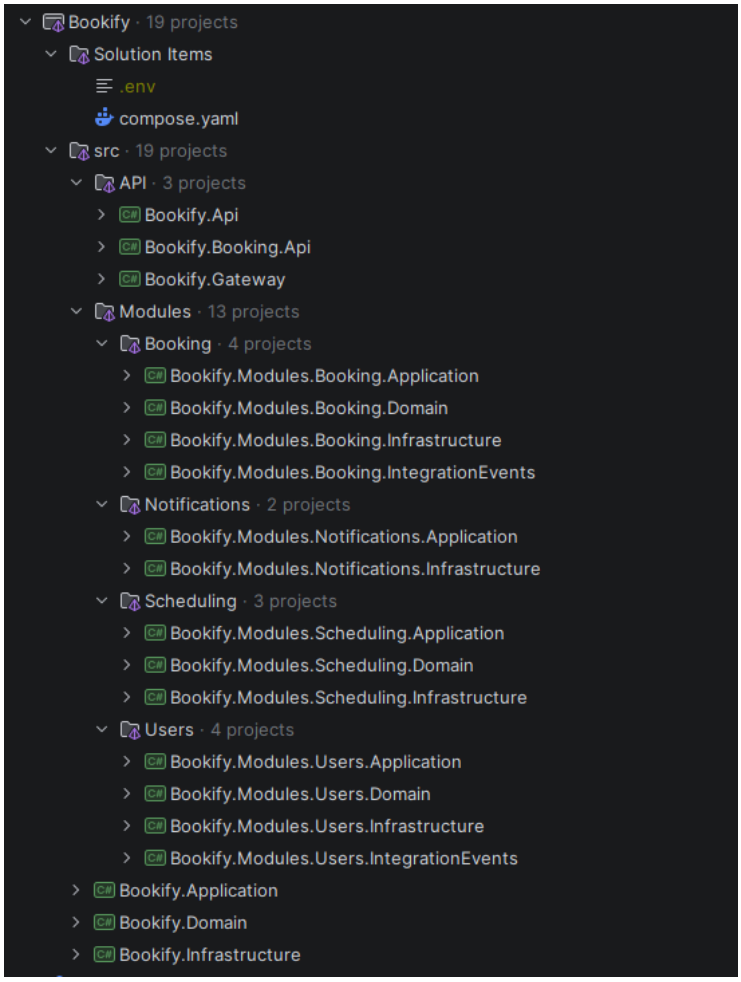


Figure 1. Bookify Solution

5.1 Users Module

The User's module covers the user management, registration, authentication and authorization. It also provides a central place to store all the user's information; it also acts as an identification mechanism for the remaining modules of the system.

The ASP.NET Core Identity was chosen as a base to perform the User management, this provided an option to use already defined mechanism of managing accounts, roles, security token, email verification, and so on, without the need to implement it by itself. The core entities can be seen on Figure 2.

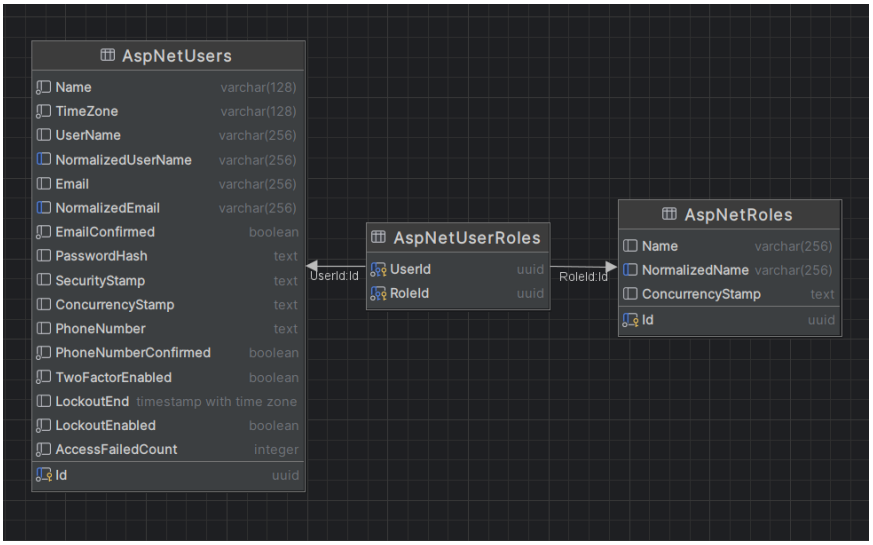


Figure 2. Users entities

5.1.1 User Registration

User registration starts when the user sends his information such as his name, email address and his time zone. Once this information is validated a new user account is created.

On the Figure 3 after a successful registration, the module creates an email confirmation token. This module does not send emails directly but publishes an UserRegisteredIntegrationEvent with user email address and the confirmation token.

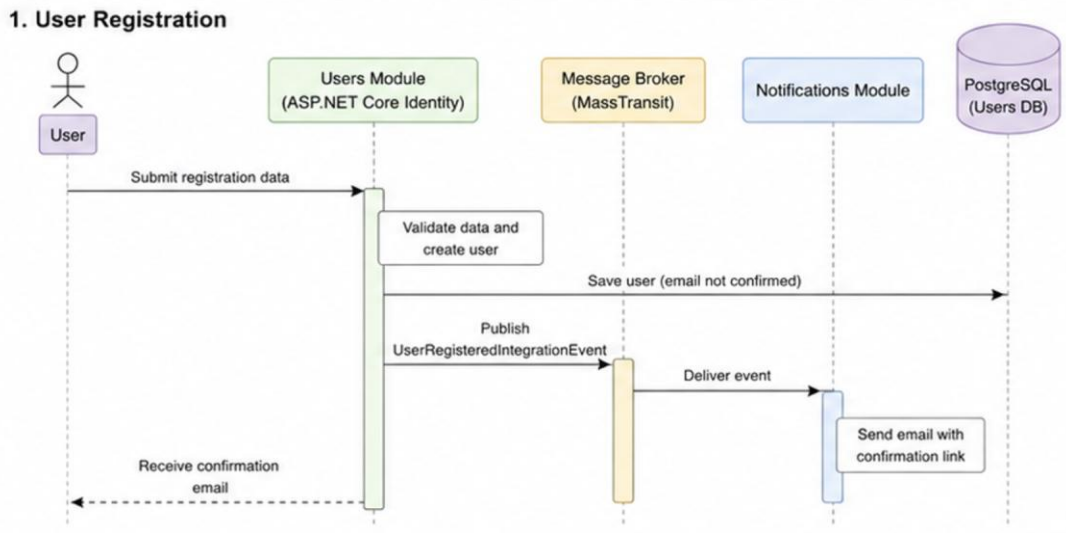


Figure 3. User registration flow

5.1.2 Email confirmation

Upon receiving the email, the user must click on the confirmation link. As can be seen on Figure 4, after the module checks the token, it enables the account. It is not possible for the user to use the login functions until the email verification is done. This method ensures that the email address belongs to the user.

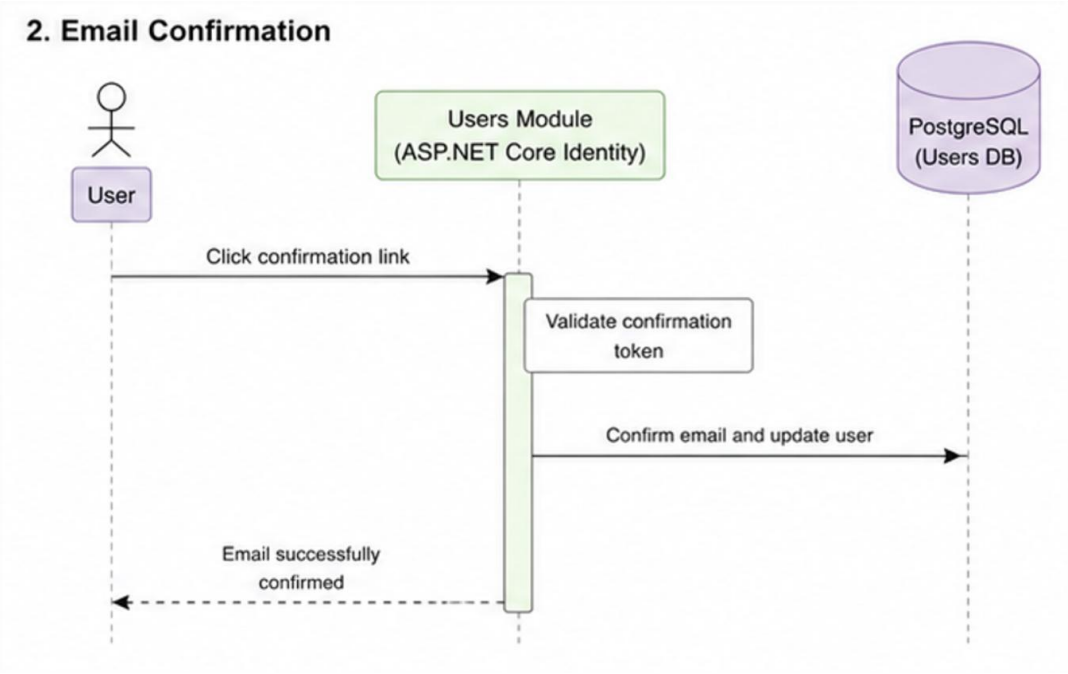


Figure 4. Email confirmation flow

5.1.3 Login without password

A different login mechanism from the password based one was utilized when building the system-password less. As can be seen on Figure 5 upon being requested to log in the user submits his e-mail and a one-off token is created by the system, and a LoginRequestedIntegrationEvent is published. The Notifications module sends an email with a link to log in to the user.

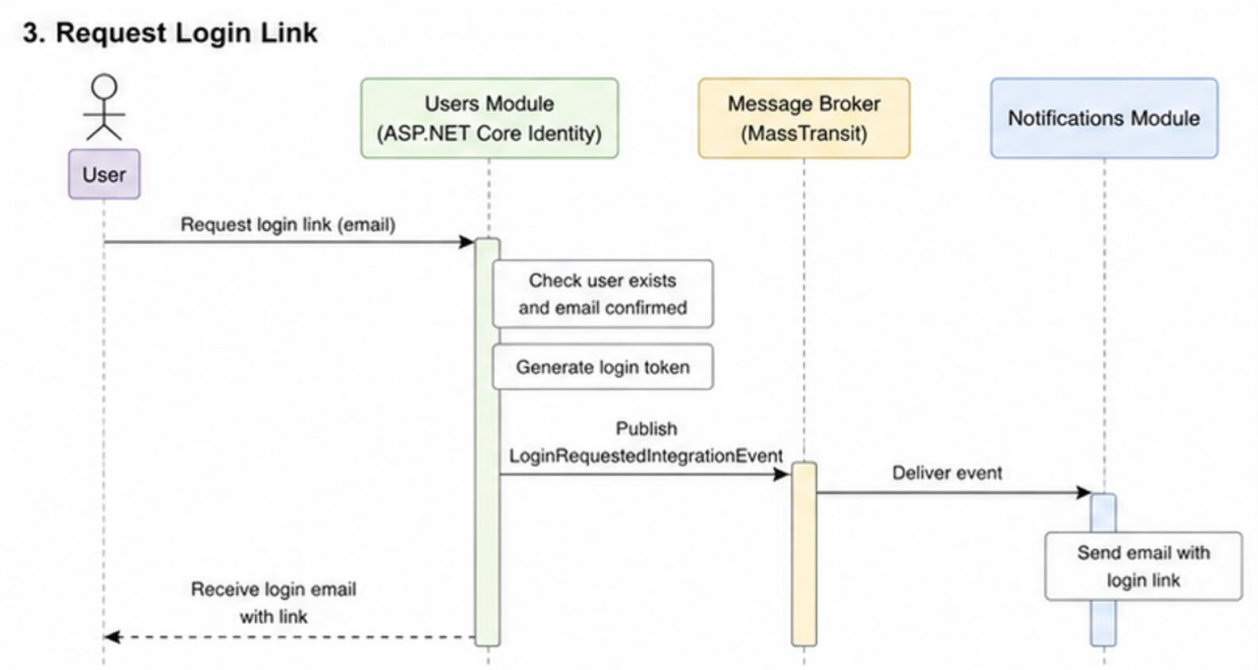


Figure 5. Request login link flow

As can be seen on Figure 6 after the link is clicked, the token is verified and the user is provided with a JWT token to keep on using the system.

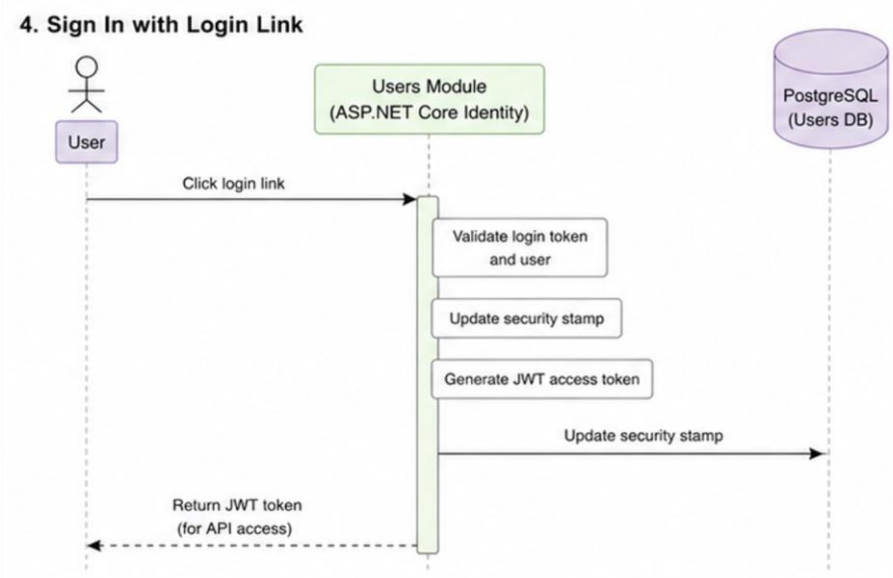


Figure 6. Sign in with login link flow

5.1.4 Integration with Notification module

One of the important architectural decisions was to decouple user management functionality from the functionality of sending notifications.

No code that sends email is included in the Users module and no email service specific dependency is added. The Users module publishes integration events, and these events are handled by the Notifications module. This decreases module coupling and follows the event driven architectural style.

5.1.5 Analytics of architectural solution

With communication over integration events, it was possible to further separate responsibilities between the modules and reduce direct dependencies. Now only the user's module will handle user account operations, and the notifications module will be responsible for message sending.

With separation of these responsibilities the system would be easier to maintain, and this matches the research primary objective of making the architecture ready to progressively migrate the monolith to microservice architecture. In case it would be needed it would be simple to extract the Notifications module in a separate service with a few to no modification to the business logic of the system.

5.2 Scheduling module

The Scheduling module controls the booking availability and delivers the available booking times. The Scheduling module is one of the key components of the scheduling system and decides which time slots can be booked.

The primary function of the Scheduling module is to compute the available slots according to rules, exceptions and bookings: rules represent recurring schedules and exceptions are specific dated occurrences.

5.2.1 Configuring event types

The module uses the Event Type entity to represent available service/appointment booking: the important properties for event type are stored such as the Name, Description, Duration, Price and owner's Time Zone.

As can be seen on Figure 7 apart from basic property it includes availability Rules with a repeating schedule, and Date Overrides where the repeating schedule can be overridden for specific dates. Additionally, it contains a list of booked time slots (Booked Slots), which are used for calculation of availability.

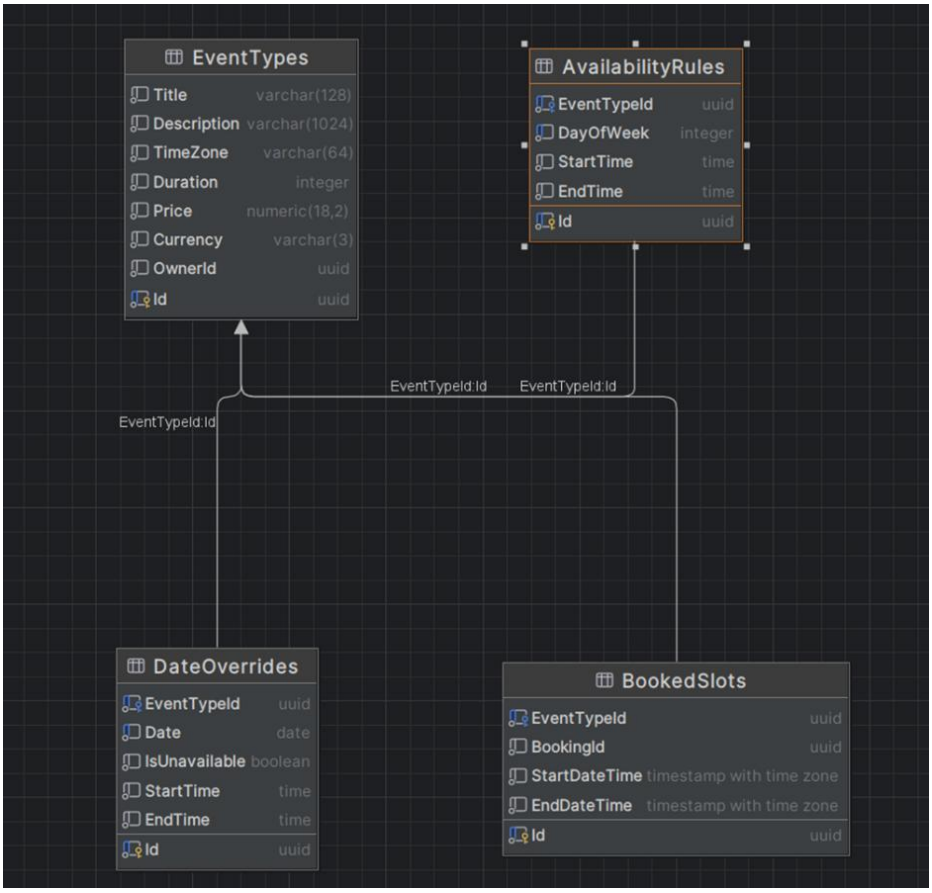


Figure 7. Scheduling entities

5.2.2 Calculating availability of slots

When a request for the list of possible free time slots is made, the module goes through the time-tables one by one. It loads the "default" availability rules for the chosen event, then it looks up for

exception periods for individual dates, which could have priority and supersede entirely availability or existing timetable slots. Finally, as can be seen on Figure 8 it checks for occupied times by bookings that already exist and takes them out of the possible slot. This way, it works correctly even with a great number of modifications and exceptions.

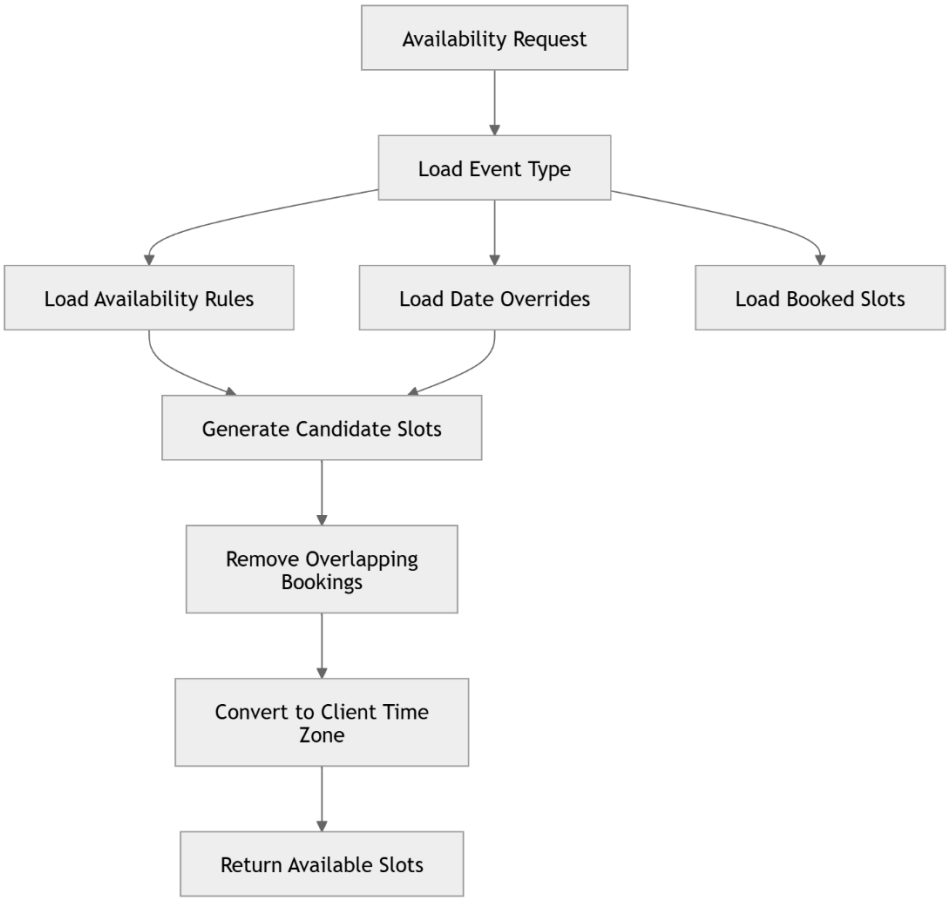


Figure 8. Slot availability flow

5.2.3 Working with Time Zones

One important thing which this module handles is time zones. It is important to know that when a user creates an event and the user making a booking may be in different time zones, so the time being shown should adjust for the respective user.

All the conflict checks happen in UTC to prevent bugs with the time zones. Once the availability has been checked the resulting available time slots are converted to the client's local time zone before the result is returned.

This approach ensures consistent data storage and the correct display of time, regardless of the users' locations.

5.2.4 Synchronization with Booking Module

The interaction between the Scheduling module and Booking module happens via integration events. As can be seen on Figure 9 when a booking is created, the Booking module fires an Order-CreatedIntegrationEvent including details of the time slot that was booked.

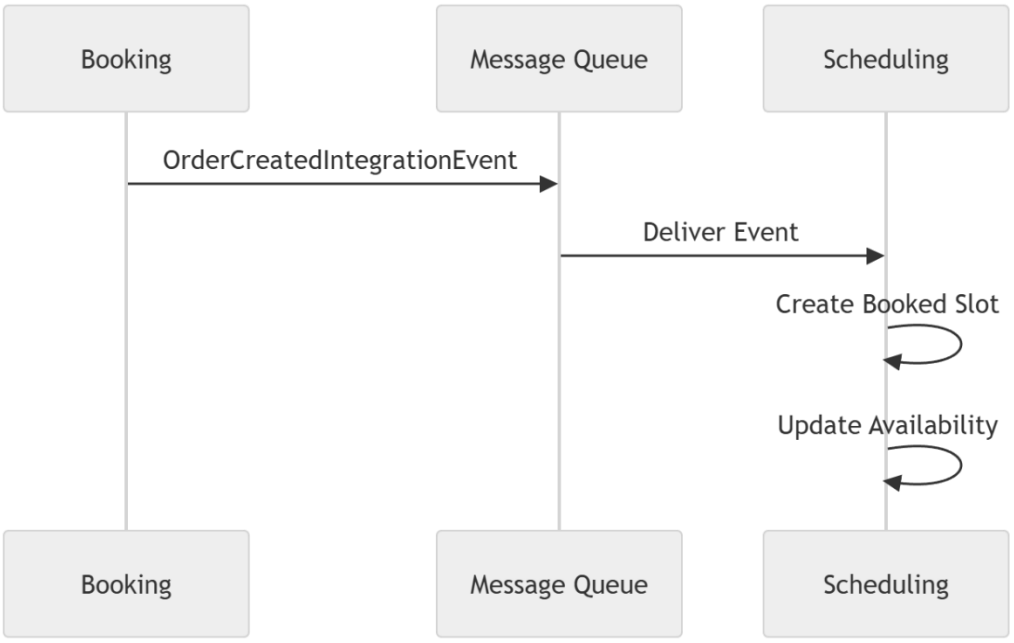


Figure 9. Synchronization flow

Once the Scheduling module receives this event, it registers a record for the time slot that has been booked. The booking causes the booking to be implicitly unavailable when it is requested again.

5.2.5 Analytics of architecture solutions

The Scheduling module is responsible for encapsulating all business logic relating to schedules and availability management of timeslots. By splitting out this responsibility into its own module, a typical pitfall of the traditional monolithic architecture-intermingling of booking, user and scheduling logic-has been avoided.

The combination of standard rules of availability, date specific overrides and event-based synchronization with the booking module gives rise to a powerful planning system that will cope with many use cases. Simultaneously, use of integration events serves the design consideration of laying groundwork for a possible migration from modular monolith to microservice architecture.

A current disadvantage is the real time calculation of availability for each request. If events and booking number scale dramatically, further optimization techniques, like caching or pre-generation of schedule, will need to be put into place.

Using an event-driven interaction model avoids direct dependencies between modules and results in a less tightly coupled system.

5.3 Booking Module

The Booking module will take care of making the bookings, processing the payment and confirming the order. As can be seen on Figure 10 it is the heart of the booking business process and binds the time slot booking availability with the payment processing and sending of the notifications.

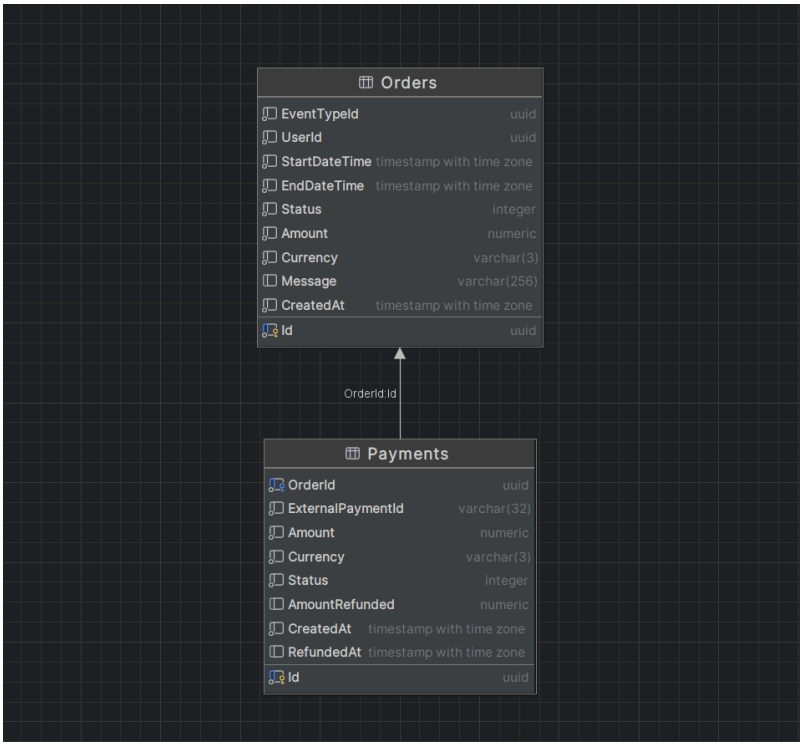


Figure 10. Booking entities

The objective of this module is that only when payment has been successfully processed will the booking be generated and confirmed. The functionality of payment will be done using the Stripe payment processing, for which an existing system can be found.

5.3.1 Booking Workflow

As can be seen on Figure 11 when a time slot is selected (is available), an order with 'Pending' state is generated and an OrderCreatedIntegrationEvent is published, which will be subscribed by Scheduling module and be able to occupy the slot.

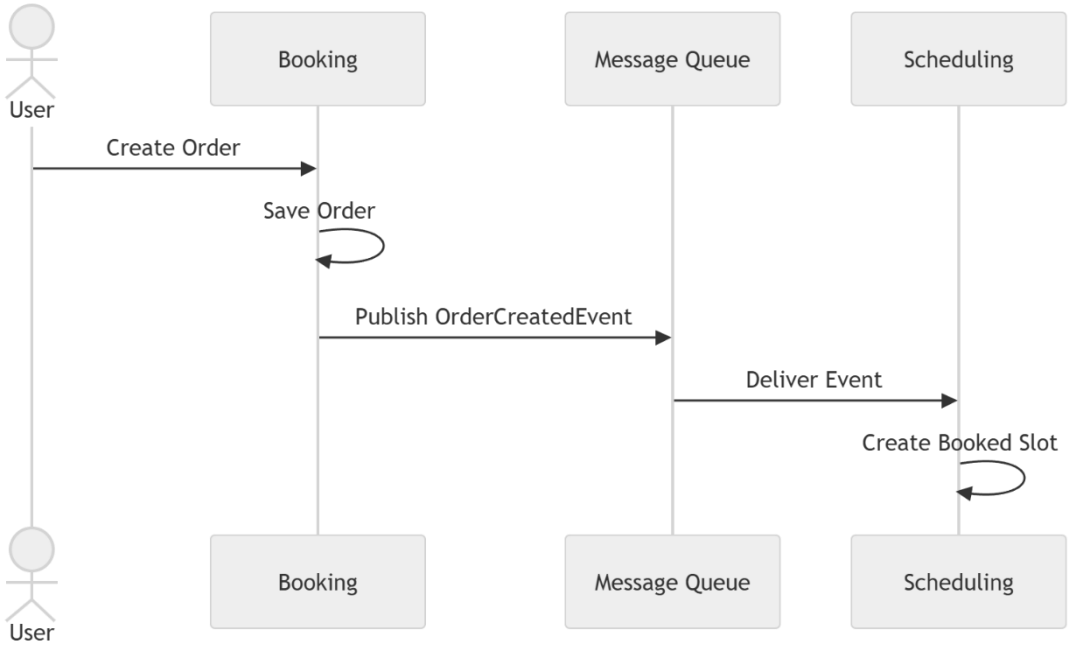


Figure 11. Booking creation flow

We are processing payments through Stripe platform. After a customer order something, the booking module establishes a payment session on Stripe API and obtains a payment link.

After a payment is fully processed, as can be seen on Figure 12 the system should update the status of the order and publish the OrderPaidIntegrationEvent. The Notifications module should be the subscriber of that integration event to send a payment confirmation email to the user.

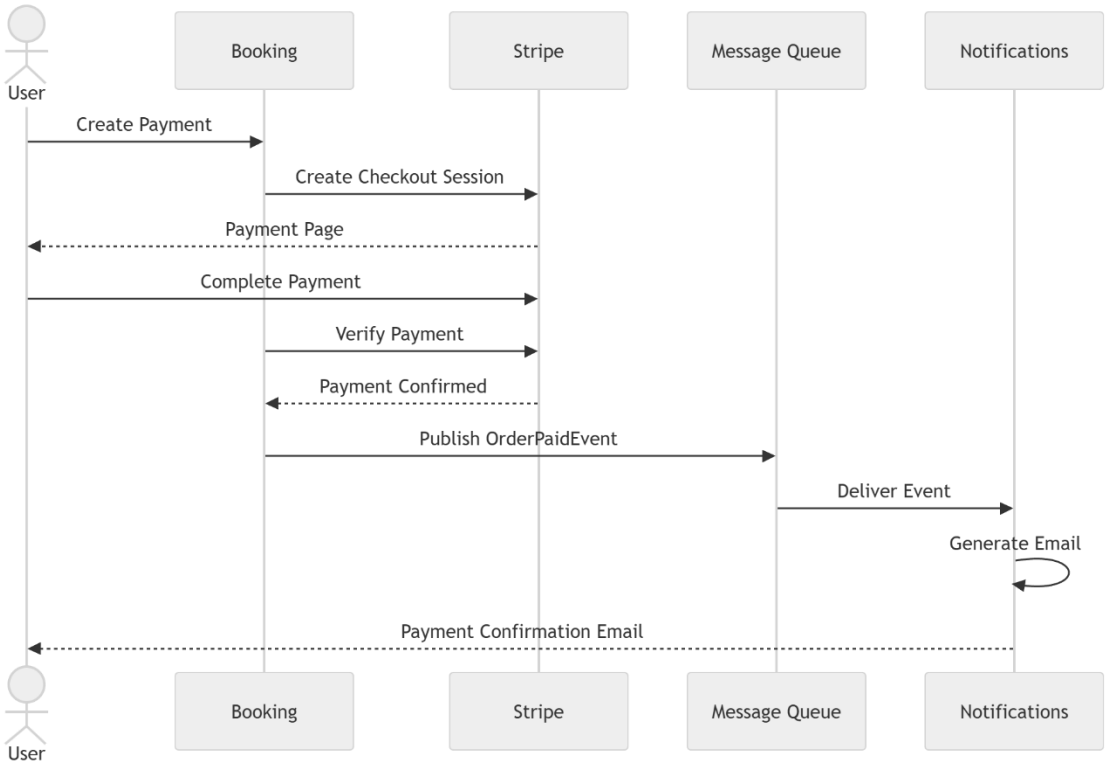


Figure 12. Booking payment flow

5.3.2 Analytics of architecture solutions

The Booking module shows communication with other parts of the system using events. As soon as the order is placed the Scheduling module gets to know that a new booking is added by order CreatedIntegrationEvent and after payment the Notification module gets notified with Order-PaidIntegrationEvent to send confirmation to the customer.

With this structure, the business logic is contained within each module. The Booking module will have no knowledge about schedule management or sending the Notification which results in loosely coupled systems that can be separated into independent microservices in the future.

5.4 Notifications Module

The Notifications module handles the delivery of notifications to users through an email message. The main goal of this module is to abstract the logic for creating and sending the messages away from other business modules. As can be seen on Figure 13 the present implementation addresses events for user registration, authentication and order payment.

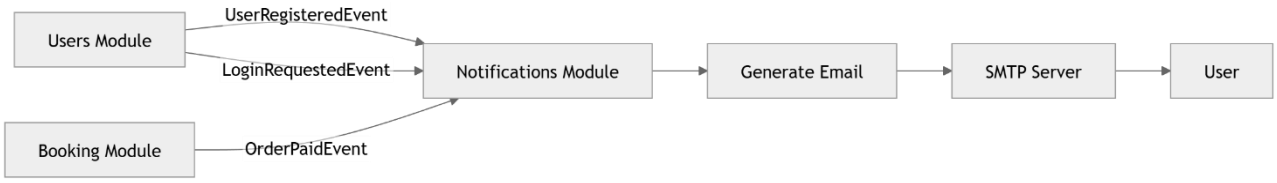


Figure 13. Notification flow

MailKit library was used to deliver messages, which is used for interacting with the SMTP servers. The actual mechanism for delivering emails is hidden in the sender service, which provides a simple interface to send emails, without caring about their content. This solution provides an interface for business logic to not care about the details of the interaction with the email infrastructure. Splitting notifications to a separate module gave us the possibility to put all message sending logic in one place. Thanks to the usage of integration events, Users and Booking modules are not aware of any notifications delivering mechanism and does not carry any logic for creating email content.

With this solution, we decreased coupling between the components and followed the single responsibility principle. Each component is only responsible for its business logic. In the same way this design eases an addition of feature in the system: we could, in the future, add SMS and push notification support, or support of an existing message sending service, etc., without having to modify the whole system.

5.5 Evolution from Modular Monolith to Microservices

5.5.1 Initial Architecture: Modular Monolith

At first the system was designed as a modular monolith. As can be seen on Figure 14 all the modules run in a single process, contained within a single Bookify.API application. Modules were communicating by calling public services on other modules. That gives a clear division of concerns but establishes tight couplings between the modules.

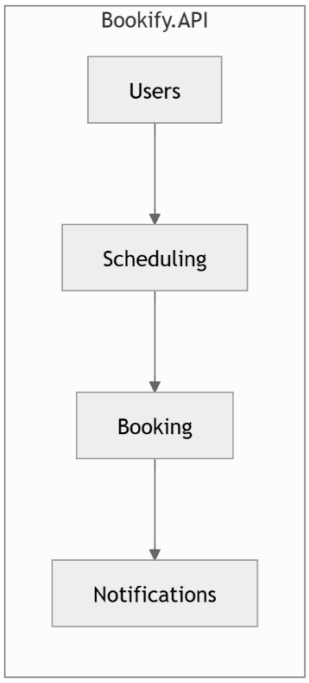


Figure 14. Initial modules

One problem with this method is the high coupling between the modules, since changes of an interface can have consequences for others.

5.5.2 Introduction of Event-Driven Communication

Coupling between modules was lowered through introduction of integration events support and RabbitMQ message broker. Instead of making direct service calls services can now "broadcast" an event which other services may subscribe to. For example:

1. OrderCreatedIntegrationEvent is used to reserve a time slot
2. OrderPaidIntegrationEvent is used to send notifications to the user
3. UserRegisteredIntegrationEvent triggers the sending of a confirmation email

This has allowed us to achieve a great level of decoupling among system components: the event source module does not rely on message listeners (it does not need to know their implementation details) as can be seen on Figure 15.

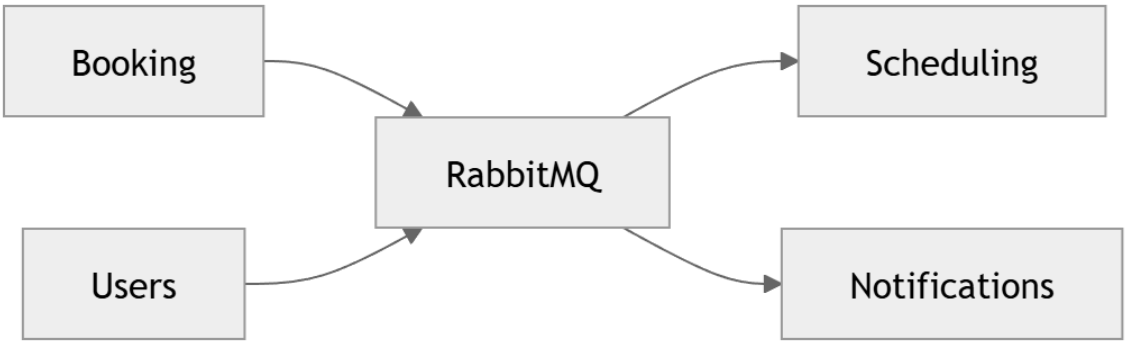


Figure 15. Event driven communication

In addition to the benefits mentioned above, message queues have enabled asynchronous processing of certain tasks, thereby increasing the system's performance in terms of user response time, and enhancing system fault tolerance.

Yet, there is still no way of completely removing synchronous communication. For example, requesting data about a specific user or event needs an instant response and hence this task will keep on using synchronous requests.

5.5.3 Extracting the Booking Module into Microservice

The next step in evolution of system was separation of the Booking module into separate microservice. As can be seen on Figure 16 for this purpose, a new project was created, Bookify.Booking.API, containing controllers and entry points for booking operations. In the meantime, all references to the Booking module in the Bookify.API project were removed.

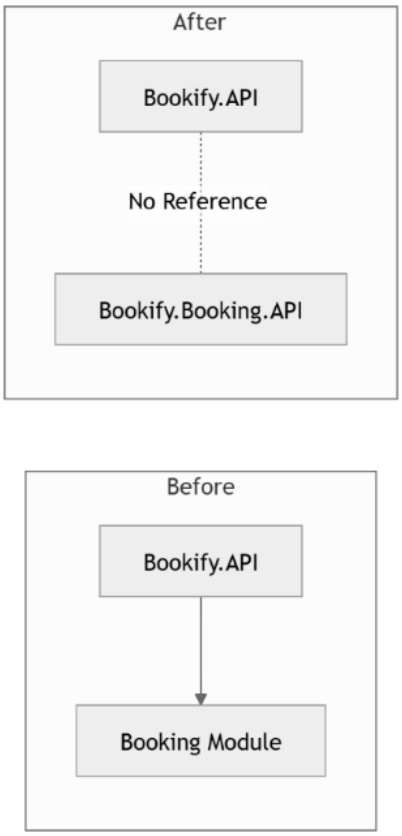


Figure 16. Module extraction

This change represents a significant architectural step, as services can now be developed independently of one another and deployed separately.

5.5.4 Introducing an API Gateway

After the split-off of the microservice, a common point of entry into the system was necessary for the customers. Thus, as can be seen on Figure 17 an API Gateway was added, built on YARP (Yet Another Reverse Proxy). Incoming requests will be handled by the gateway and routed to the target service:

1. Requests to /api/users/* are routed to Bookify.API
2. Requests to /api/orders/* are routed to Bookify.Booking.API

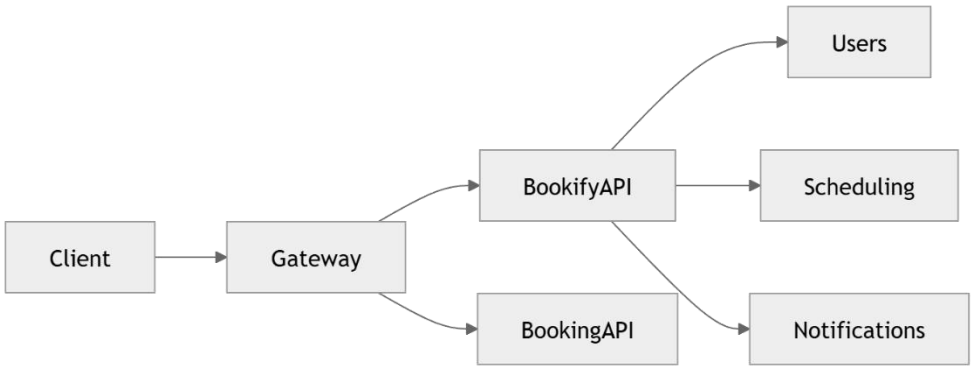


Figure 17. Gateway introduction

The API Gateway will enable you to abstract the internal structure of your system from clients and act as one single point for authentication, authorization and request routing.

6 Evaluation and Results

6.1 Scalability Analysis

Some benefits of introducing Booking as a separate microservice over a modular monolith are as follows. First, as can be seen on Figure 18 it is possible to scale Booking services independently of the remaining parts of the system. If there is an increase in the number of requests in the orders creation or the payments process, it is possible to launch more instances of Bookify.Booking.API without impacting Users, Scheduling and Notifications.

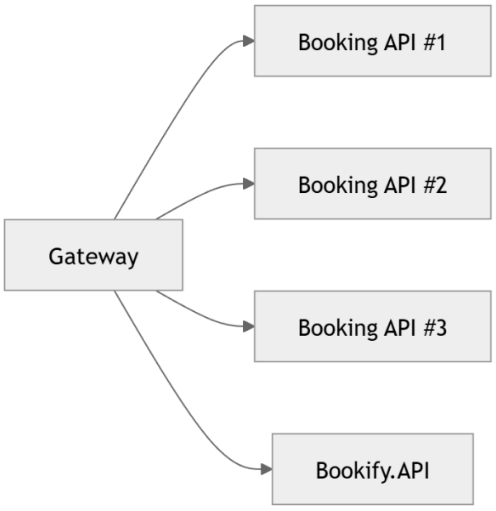


Figure 18. Multiple Booking API instances

Secondly, using independent deployment of the booking function enables updates to be performed without redeploying the whole system.

Third, since the service communication was already based on events, no major change to business logic was needed to spin Booking out.

6.2 Architecture analysis

This evolution proves one of the benefits of the modular monolith approach as a transitional step. The system starts with the ease of use of a monolithic application's development and deployment, while providing good separation between parts of the system.

Using integration events from the beginning of development has an added benefit, and it is the easing of the later transition to the microservices approach. In our example, the Booking module was extracted to a separate service without significant change of business logic, because it was already interacting with other parts of the system through events.

These results confirm that modular monolith is a good transitional step and that migration to microservices architectures can be smoother from a modular monolith compared to a traditional one.

6.3 Survey analysis

To assess the suitability of the proposed architectural style, a survey of software developers was carried out. This was intended to elicit an expert opinion regarding the feasibility of using the modular monolith as an intermediate step between the monolith and the microservices architecture.

The survey included the following five developers: 1 Senior Developer, 2 Middle Developers, 2 Junior Developers. All had experience with building backend applications and understanding monolithic architecture, and three respondents had experience with microservices. They were asked the following questions.

Question 1. What architecture would you choose for a new medium-sized project?

1. Modular monolith - 4 answers (80%)
2. Microservices - 1 answer (20%)
3. Traditional monolith - 0 answers (0%)

Most of the answers favored the modular monolith, which participants believed provided a reasonable balance between the simplicity of development and ability to expand the system in the future. This verifies the chosen architectural decision.

Question 2. Does having clear boundaries between modules make the subsequent transition to microservices easier?

1. Yes - 5 answers (100%)
2. No - 0 answers (0%)

Every participant had the same opinion that well-defined boundaries between modules reduce the complexity of future migrations. The conclusion supports the practice part of the research where the Booking module was separated as a separate service without considerable business logic changes.

Question 3. Which key advantage of a modular monolith do you consider to be the most important?

1. Improving system maintainability - 2 answers (40%)
2. Simplifying the transition to microservices - 2 answers (40%)
3. Ease of deployment compared to microservices - 1 answer (20%)

It's obvious from the answers that developers don't only see the modular monolith as a code organizing pattern but also as a tool to incrementally transform the architecture. What's interesting, too, is that most developers answered with features that can be seen as directly linked to the aims of this investigation.

Question 4. What is the most challenging issue when transitioning to microservices architecture?

1. Increased complexity in system maintenance and operation - 2 answers (40%)
2. Interaction between services - 2 ответа (40%)

3. Ensuring data consistency - 1 ответ (20%)

It can be said from the obtained answers that the critical issue is not business logic implementation, but distributed systems specifically. It was also found in a study which claims that after spin-off of Booking as an independent service, it was essential to add message queue, API Gateway and others for inter-service communication.

Question 5. Do you consider event-driven communication to be useful when preparing a system for the transition to microservices?

1. Yes - 4 answers (80%)
2. Partially - 1 answer (20%)
3. No - 0 answers (0%)

The most participants indicated that use of integration events allows lowering of coupling between system parts, as well as eases future decomposition into separate services. It has also been demonstrated by a practical implementation where modules communication by means of RabbitMQ permitted to pull out Booking module with minimum modification of the current system code.

The result from the survey then supports the conclusions drawn from the study. A large majority of the developers surveyed believe that the modular monolith is a viable approach for small and medium projects and that it is an appropriate stage on the road to microservices architecture. Other important findings also indicated that clear module separation and usage of integration events was one of the most important reasons in a migration.

6.4 How does a modular monolith help to structure a system before transitioning to microservices?

The result of implementation proved that a modular monolith could act as an intermediary stage from a monolith into a microservices application. By separating the system into distinct, self-contained modules, individual business logic can be separated and the coupling between the different parts of the system decreased.

Four modules were designated in the created system: Users, Scheduling, Booking, and Notifications. Each module includes its own business logic, domain objects and infrastructure. This separation has made the system much more maintainable and laid the foundation for individual modules to be subsequently extracted as separate services.

From these results we can conclude that the modular monolith is able to obtain a good part of the advantages offered by the microservices architecture at the organizational level of code but at the same time keeps all the simplicity of deployment and operation of the monolith application.

6.5 What principles are used to identify modules with clearly defined responsibilities?

Modules are defined around the business capabilities of the system and not around the technical concerns. The function of each module relates to one specific aspect of the business and encapsulates the business processes that deal with this business concern.

Users' module handles user management and authentication, scheduling schedules and event availability, booking handles bookings and payment, and Notifications handles the notification service for sending messages.

This design relates to the notion of bounded contexts defined in Domain Driven Design. This in turn causes modules to be highly cohesive in their internal composition and lowly dependent on the outside world. This leads to less complexity for developers and fewer side-effects when modifications are made.

6.6 How is the phased migration of an individual module to a microservice carried out?

The Booking module was also migrated experimentally into a separate microservice as part of the study.

In the first step, the modules remained within a single application. The interaction between modules was improved by adding integration events and a RabbitMQ message queue. This made modules more independent from each other and brought the system closer to the ability to operate independently.

At the next stage, a separate project Bookify.Booking.API was created for the Booking module. The references to the module were deleted from the Bookify.API project, while communication with other parts of the application was maintained through HTTP API and integration events.

It was noticed that defined modules with events and integration events allow easier moving a part of the system to a new independent service without major rewrite of business logic.

6.7 What challenges arise when transitioning partially to a microservices architecture?

Although, individual services can bring advantages when it is split out from the system, hybrid architecture does not only have these benefits, but also poses the following difficulties:

Firstly, complexity increases with respect to architecture. System complexity will increase since we need to take care of multiple processes, communication between processes rather than single application.

Secondly, failure points may increase. Network failure, a specific service is unavailable, or message delivery delay may affect business processes from execution.

Thirdly, diagnosis and debugging become more difficult. To process a single business process, a business process might cross different applications and message queues, additional monitoring and tracing tools are required.

In conclusion, although the conclusion confirms that microservice architecture provides additional possibilities for scalability, it will be at the price of the complexity in developing and maintaining the system.

6.8 How is interaction between the monolithic and microservice parts of the system ensured?

Within the presented architecture, the Main application and the separate Booking service can communicate using two mechanisms.

For all synchronous operations (where immediate response is needed) communication happens using HTTP requests through the API Gateway, which is implemented by YARP. The Gateway offers a single point of access to all services and masks the underlying complexity.

The events communication is done through RabbitMQ. After the creation of the order an event OrderCreated will be published. The Scheduling module subscribes to this event to book the slot for the order. After the payment is completed an OrderPaid event will be published by Main module, subscribed to the Notification module for the purpose of sending the mail to the customer.

The mixed synchronous-asynchronous communication allows us to achieve a coherent behavior of the application after splitting the Booking functionality.

6.9 Overall assessment of the results

Results the study confirmed that a modular monolith is a well-suited architectural style for systems that might require to become a microservice architecture in the future. It was demonstrated that clear separation of responsibilities in between modules allowed minimal rework for spinning out the Booking module to be separated service.

This experiment showed that the use of integration events and boundaries between modules would allow for a smooth transition without too much refactoring of business logic in the application. At the same time this experiment demonstrated that microservices cannot be a "one-fits-all" architecture. While microservices would allow for scaling and individual deployment, it is complicated and requires more for dealing with distributed parts of the system.

Therefore, small to medium-sized projects are well worth considering running on a modular monolith architecture, as it is a good middle-way between a monolith and microservices.

7 Conclusion

7.1 Reflection

It has been proved by the experiment that the modular monolith can be considered a good stepping-stone on the road from the traditional monolith to microservices. Since the system was developed practically, the opportunity arose to analyze the positive sides of this architecture and at the same time identify the downsides of transition to the distributed architecture. A sustainable development perspective of the modular monolith is to better utilize organizational and technical resources. Rather than experiencing initial operational overhead and cost of implementing microservices infrastructure, a gradual system evolution can occur while ensuring developer productivity is sustained. This approach helps to provide an economically sustainable solution by avoiding migration risks and unnecessary upfront expenditure of distributed infrastructure. In addition, explicit boundaries among modules enhance communication between development teams and the maintainability of the system for future growth, which aids social sustainability in software engineering development.

The major conclusion drawn from the research was that the successful transition to microservices depends on the quality of the architectural boundaries between the functional parts of the system rather than on the number of individual services. Due to the clear division of functionality between the User, Scheduling, Booking and Notifications modules, the Booking module could be easily separated into an independent service without making much change to the business logic.

The research also revealed that the introduction of integration events at the very early stages of the project helps a lot when the further development of architecture occurs. At the same time, it has been found that transition to microservices always leads to the increased complexity of the system and therefore to an urgent need for additional solutions for its monitoring, error handling and interaction control between distributed components.

7.2 Practical Recommendations

From the conclusion of this study, several practical suggestions can be provided for software developers and architects.

A microservices architecture should not be adopted early in a system's lifecycle unless there is a clear need for it. A modular monolith appears to offer a better trade-off between development ease and maintenance burden in small and medium-sized teams.

Boundaries for modules should be drawn based on business domains or functionality as opposed to technical parts of the system so that these parts of the system can later be decoupled into a microservice.

By applying event-driven communication between modules, the system can be predisposed to the possibility of future migration, and this will minimize the dependency between modules.

It should be confirmed whether a component needs to be separated into a microservice due to independent scalability, performance or deployment requirements before separation takes place.

7.3 Future Work

Despite the attained results, the research could be expanded further in several ways.

First, it would be worthwhile splitting further system modules into standalone microservices and investigate how this migration affects the system's performance, fault tolerance and maintainability.

Secondly, further research could investigate the use of distributed monitoring and request tracing frameworks such as OpenTelemetry to facilitate the debugging of distributed systems.

Thirdly, one area that appears very interesting for further research is the study of methods to guarantee data consistency across services by applying Saga and Outbox patterns.

Lastly, it would be worth researching the use of quantitative metrics such as development time, maintenance cost, performance, and complexity for the deployment and performance of a modular monolith compared to a full-blown microservice architecture.

References

Al-Debagy, O., & Martinek, P. (2018). A comparative review of microservices and monolithic architectures. In 2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI) (pp. 149–154). IEEE. https://doi.org/10.1109/CINTI.2018.8928192

Amazon Web Services. (n.d.). *What is unit testing?* https://aws.amazon.com/what-is/unit-testing/

Blinowski, G., Ojdowska, A., & Przybyłek, A. (2022). Monolithic vs. microservice architecture: A performance and scalability evaluation. *IEEE Access, 10*, 20357–20374. https://doi.org/10.1109/ACCESS.2022.3152803

Faustino, D., Gonçalves, N., Portela, M., & Rito Silva, A. (2024). *Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation*. Performance Evaluation, 164, 102411. https://doi.org/10.1016/j.peva.2024.102411

Fowler, M., & Lewis, J. (2014, March 25). *Microservices: A definition of this new architectural term*. martinfowler.com. https://martinfowler.com/articles/microservices.html

Kalske, M. (2017). *Transforming monolithic architecture towards microservice architecture*

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

[Master's thesis, University of Helsinki]. Helda Digital Repository. http://hdl.handle.net/10138/234239

Microsoft. (n.d.). *.NET | Free. Cross-platform. Open source*. https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet

Microsoft. (2024). *Integration testing*. Engineering Fundamentals Playbook. https://microsoft.github.io/code-with-engineering-playbook/automated-testing/integration-testing/

Moraes, R., Silva, A. R., Portela, M., & Gonçalves, N. (2025). *Modular monolith architecture in cloud environments: A systematic literature review*. *Future Internet, 17*(11), 496.

Newman, S. (2019). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Newman, S. (2020). *Monolith to microservices: Evolutionary patterns to transform your monolith*. O'Reilly Media.

RabbitMQ. (2026). *RabbitMQ documentation*. https://www.rabbitmq.com/